

余白博客

当前项目使用报告

功能、架构、本地开发、AI 接入、生产部署、安全、备份、监控与当前交付状态

报告日期: 2026-07-29

代码分支: `codex/blogdemo`

基线提交: `9e9ab2e`

生产域名配置: `https://hxn timer .top`

状态说明：本报告基于当前本地工作区。OpenCode 集成尚未提交、推送或部署，因此不代表线上网站已经接通 OpenCode。

一、项目概况

余白是一个前后端分离的个人博客与私人内容管理系统，包含公开博客、学习笔记、美食菜单、知识图谱、内容后台、双角色账户、AI 助手及生产运维设施。



二、技术栈

层级	技术
前端	Vue 3.5、TypeScript 5.9、Vite 8、Pinia、Vue Router、Axios
编辑器	Tiptap 3、Markdown、KaTeX、Lowlight
PWA	vite-plugin-pwa、Workbox
后端	Java 21、Spring Boot 3.5.16
数据层	PostgreSQL、Spring Data JPA、Flyway
安全	Spring Security、JWT、Refresh Token、BCrypt、TOTP
缓存与监控	Caffeine、Actuator、Prometheus
测试	JUnit、Testcontainers、Vitest、Vue Test Utils
部署	Nginx、systemd、GitHub Actions、rsync
AI	OpenAI Compatible API、OpenCode Server sidecar

三、用户与权限

角色	能力
游客	首页、文章、系列、标签、归档、美食、关于、搜索
PARTNER	登录、账户管理、私人笔记访问、今日菜单和用餐记录
ADMIN	全部功能，包括内容、AI、附件、音乐、语录、统计管理

ADMIN 具有所有系统权限；PARTNER 仅具有账户和厨房相关权限，无法进入内容或 AI 管理后台。

四、主要功能

公开内容

- 文章列表、详情、分类、标签、系列和归档。
- 文章点赞、浏览统计、Markdown 内容展示。
- 全站搜索，按文章、笔记和菜品分组。
- 美食菜谱、食材、步骤、收藏、评分与深链打开。
- RSS、动态 Sitemap、robots.txt。
- SEO Meta、Canonical、Open Graph、JSON-LD。
- PWA 安装、自动更新和图片运行时缓存。

登录功能

- 学习笔记和知识图谱。
- 账户资料、密码修改和 TOTP 两步验证。
- Refresh Token 恢复登录状态。
- 今日菜单、用餐打卡、历史记录和菜品统计。

管理后台

- 文章、菜品、笔记、系列的增删改查。
- 草稿、发布、撤回、归档和版本冲突处理。
- Markdown 导入、导出，以及图片粘贴、拖放和附件管理。
- 音乐曲目、每日语录、Dashboard 统计和趋势。
- AI 供应商、模型、限额、连通测试、用量统计和 AI 对话。

五、本地运行

前置要求

- Java 21、Maven、Node.js 22.13 或更高版本、npm、PostgreSQL。
- 可选 Docker，用于 Testcontainers。
- 可选 OpenCode，用于 AI sidecar。

初始化数据库

```
psql -U postgres -f backend/database/bootstrap.sql
Copy-Item backend/.env.example backend/.env.properties
```

必须修改 `backend/.env.properties`：

```
DB_PASSWORD=数据库密码
APP_JWT_SECRET=至少32位随机字符串
APP_ADMIN_USERNAME=管理员用户名
APP_ADMIN_PASSWORD=强密码
APP_JWT_COOKIE_SECURE=false
```

启动服务

```
cd backend
mvn spring-boot:run

cd frontend
npm install
npm run dev
```

前端地址为 `http://localhost:5173`，后台入口为 `http://localhost:5173/admin/login`，后端健康检查为 `http://localhost:8080/actuator/health`。

六、数据库

数据库由 Flyway 管理，应用启动时自动迁移。Hibernate 使用 `validate`，不会自行修改表结构。

当前最新迁移为 `V30__add_ai_providers_provider_type.sql`，为 AI 供应商增加 `OPENAI_COMPATIBLE` 和 `OPENCODE_SERVER` 两种协议。

本机数据库记录不会自动同步到生产数据库。代码部署后，生产数据库会执行结构迁移，但供应商等业务记录必须在生产后台单独创建。

七、AI 使用方式

类型	用途	密钥位置
OPENAI_COMPATIBLE	DeepSeek、OpenAI、Kimi、智谱、通义、Ollama 等	AES-256-GCM 加密后存入数据库
OPENCODE_SERVER	本机 OpenCode sidecar	OpenCode 认证目录和服务器 env

保存 OpenAI Compatible 密钥需要配置：

APP_AI_MASTER_KEY=至少32位随机字符串

OpenCode 不把供应商 API key 存入博客数据库，也不需要后台填写 API key。

生产后台 OpenCode 配置

字段	值
名称	OpenCode DeepSeek
协议类型	OpenCode Server（内建）
Base URL	http://127.0.0.1:4096
模型列表	deepseek-v4-flash
默认模型	deepseek-v4-flash
启用	是
API 密钥	不填写

保存后点击“测试连通”，成功后设为默认供应商。

八、OpenCode 架构

```
yubai-blog.service
| HTTP Basic
▼
127.0.0.1:4096
yubai-blog-opencode.service
| HTTPS
▼
opencode-go/deepseek-v4-flash
```

核心配置：

```
/etc/yubai-blog/app.env
/etc/yubai-blog-opencode/opencode.env
/etc/yubai-blog-opencode/opencode.json
/var/lib/opencode
```

后端和 sidecar 使用相同的随机 Basic 密码，但分别读取隔离的 env 文件。默认 agent 为 `blog-ai`，provider ID 为 `opencode-go`。

首次认证

```
sudo -u opencode HOME=/var/lib/opencode opencode auth login
```

启动与检查

```
sudo systemctl start yubai-blog-opencode.service
sudo systemctl restart yubai-blog.service
sudo systemctl status yubai-blog-opencode.service
curl http://127.0.0.1:8080/actuator/health
```

九、生产部署

GitHub Actions Deploy workflow 可由 `v*` 标签或手动触发，并受 GitHub `production` Environment 审批保护。

1. 执行前后端测试和构建。
2. 生成前端静态产物和后端 JAR。
3. 通过 rsync 上传到新 release 目录。
4. 原子切换 `/opt/yubai-blog/current` 软链接。
5. 重启后端并进行最长 60 秒健康检查。
6. 失败时自动回滚；成功后保留最近五个 release。

当前限制：GitHub Actions 只发布前端和后端产物，不会安装 OpenCode、更新 systemd unit、部署 `opencode.json`、创建 `sidecar` 目录或修改生产 env。现有服务器第一次接入 OpenCode 时必须手动完成服务器侧安装。不要直接重跑 `bootstrap-server.sh`，因为它会在发现已有生产 env 或数据库时拒绝覆盖。

十、Nginx 与网络

生产 Nginx 已配置 HTTPS 强制跳转、TLS 1.2/1.3、HSTS、安全响应头、登录与 API 限流、Source map 禁止公开、SPA fallback，以及 Sitemap 和 robots 转发。

端口	用途	公网策略
80 / 443	Nginx HTTP/HTTPS	允许公网访问
5432	PostgreSQL	禁止公网访问
8080	Spring Boot	仅本机
4096	OpenCode sidecar	仅 <code>127.0.0.1</code>

十一、安全设计

- 密码使用 BCrypt；JWT Access Token 配合 HttpOnly Refresh Token。
- 密码修改后使旧 session 失效；支持 TOTP 两步验证。
- 登录 PoW、验证码、失败冷却和 Nginx 限流。
- API 角色和 capability 双重鉴权。

- 上传类型、大小和路径限制；DOMPurify 防止 Markdown/XSS。
- OpenAI 密钥使用 AES-256-GCM 加密。
- OpenCode key 不进入博客数据库，sidecar 仅监听回环地址。
- OpenCode 使用独立低权限用户，agent 全局禁止工具调用。
- OpenCode URL 仅允许 loopback，拒绝重定向、userinfo、query 和 fragment。

剩余风险主要来自服务器 root 权限被攻破、sidecar 误绑定 `0.0.0.0`、错误反向代理、调试日志泄露，以及后端远程代码执行。

此前 `opencode debug config` 的终端输出曾包含供应商 key 明文。该 key 未进入源码或 Git diff，但建议上线前撤销并轮换。

十二、备份与恢复

systemd timer 每天 03:20 执行备份，并加入最多 15 分钟随机延迟。

- 备份 PostgreSQL custom-format dump 和附件目录压缩包。
- 生成 SHA-256 校验清单，本地保留 14 天。
- 可选上传腾讯云 COS，并使用服务端 AES256 加密。
- Manifest 最后上传，作为完整批次标记。

`/var/lib/opencode` 不在备份范围。主机恢复后需要重新执行 OpenCode 登录。

十三、监控与排障

健康检查为 `GET /actuator/health`，Prometheus 指标为 `GET /actuator/prometheus`。健康接口只显示 UP/DOWN，不公开数据库或磁盘详情。

```
systemctl status yubai-blog
systemctl status yubai-blog-opencode
journalctl -u yubai-blog -n 200 --no-pager
journalctl -u yubai-blog-opencode -n 200 --no-pager
curl http://127.0.0.1:8080/actuator/health
ss -ltnp
```

如果网站正常但 AI 不可用，应依次检查 sidecar 状态、4096 监听地址、两侧 Basic 密码、OpenCode 登录状态、后台 provider、默认模型和服务器出站 HTTPS。

十四、测试状态

项目	结果
后端测试	530/530 通过
前端测试	451/451 通过
TypeScript typecheck	通过
前端生产构建	通过
Flyway/JPA 测试	通过
Shell、OpenCode JSON、git diff 检查	通过
真实 OpenCode 调用	返回 LIVE_OK

构建存在一个非阻塞警告：Tiptap chunk 压缩后超过 500 KB，只影响部分后台编辑器页面的加载体积。

十五、当前代码状态

- 28 个已跟踪文件有修改，7 个新增文件尚未跟踪。
- OpenCode 集成未提交、未推送、未部署。
- V30 尚未进入生产，生产 AI 供应商记录尚未创建。
- 线上网站目前不能认定为已经连接 OpenCode。
- backend/.env.properties 没有进入 Git 改动。
- 敏感信息扫描未发现真实凭据；唯一匹配项是测试夹具 sk-registry-key。
- README 的 AI 章节仍主要描述 OpenAI Compatible 模式，OpenCode 设计位于 docs/architecture.md。

十六、建议上线顺序

1. 在供应商后台轮换此前出现在终端输出中的 API key。
2. 审查并提交当前 OpenCode、前端和认证修复。
3. 推送分支并通过 CI。
4. 在服务器安装 OpenCode，并手动安装用户、目录、配置、env 和 systemd unit。
5. 确认 sidecar 只监听 127.0.0.1:4096。

6. 使用 `opencode` 用户完成供应商登录。
7. 发布后端和前端并执行 `V30`。
8. 在生产后台创建 `OPENCODE_SERVER` 供应商。
9. 执行连通测试并设为默认。
10. 发送真实 AI 对话，检查日志、Prometheus、备份和账单。

当前结论：项目本地实现和验证已经完成，具备安全接入 OpenCode 的代码与部署配置。生产接通仍依赖提交发布、服务器首次 sidecar 安装、OpenCode 登录和生产后台供应商配置。

报告依据：当前工作区源码、README、architecture 文档、systemd/Nginx/备份配置、GitHub Actions 工作流及本轮测试结果。